

TASK SPECIFICATION: Multi-Channel Retail Banking Core Management System (Java)

1. PROJECT OBJECTIVE & PHILOSOPHY

You are tasked with building a production-grade, fully complete **Retail Banking Core System** in Java (JDK 17+ or JDK 21+).

The Core Standard:

The goal is not superficial feature bloat, but **deep conceptual completeness and invariant enforcement**. Every Java feature used (Inheritance, Polymorphism, Abstraction, Interfaces, Collections, Object method overrides, Exception Handling, and Multithreading) must solve a concrete banking requirement. There must be zero placeholder logic, zero // TODO comments, and zero leaking abstractions.

2. PACKAGE STRUCTURE & ARCHITECTURE

Organize the project cleanly into standard layered packages:

```
src/
├── com/
│   └── bank/
│       ├── model/
│       │   ├── Customer.java
│       │   ├── BankAccount.java      (abstract)
│       │   ├── SavingsAccount.java
│       │   ├── CurrentAccount.java
│       │   ├── Transaction.java      (record or immutable class)
│       │   └── TransactionType.java  (enum: DEPOSIT, WITHDRAWAL, TRANSFER_OUT,
│       │                               TRANSFER_IN, INTEREST, FEE)
│       ├── service/
│       │   ├── Transactable.java     (interface)
│       │   ├── Reportable.java       (interface)
│       │   ├── BankService.java      (interface)
│       │   └── BankServiceImpl.java  (thread-safe service implementation)
│       ├── exception/
│       │   ├── BankingException.java (base checked exception)
│       │   ├── InsufficientFundsException.java
│       │   └── OverdraftLimitExceededException.java
```

```

|   |— AccountNotFoundException.java
|   |— CustomerNotFoundException.java
|   |— DuplicateAccountException.java
|   |— InvalidAmountException.java
|   |— concurrency/
|   |   |— ConcurrentTransactionWorker.java
|   |   |— ConcurrencyHarness.java    (demonstration of race condition prevention &
deadlock-free transfers)
|   |— ui/
|       |— BankApplicationCLI.java    (robust Scanner-based interactive CLI)

```

3. DOMAIN INVARIANTS & DETAILED SPECIFICATIONS

A. Model Layer

1. Transaction.java

- **Type:** Immutable record or class with final fields.
- **Fields:**
 - String transactionId (generated via UUID.randomUUID().toString())
 - LocalDateTime timestamp
 - TransactionType type
 - double amount
 - double balanceAfter
 - String description
- **Requirements:**
 - Formatted output in toString() resembling a bank passbook row.

2. Customer.java

- **Fields:**
 - final String customerId (e.g., "CUST-1001")
 - String fullName
 - String email
 - String phone
 - final List<String> associatedAccountNumbers (ArrayList)
- **Behavior:**
 - Method addAccount(String accountNumber) to maintain link.
 - Return unmodifiable list for associatedAccountNumbers.
 - Proper equals(Object o) and hashCode() based strictly on customerId.
 - Informative toString().

3. Transactable.java (Interface)

- Contract for core movements:

- void deposit(double amount) throws InvalidAmountException;
- void deposit(double amount, String note) throws InvalidAmountException; (Method Overloading)
- void withdraw(double amount) throws InvalidAmountException, InsufficientFundsException, OverdraftLimitExceededException;

4. Reportable.java (Interface)

- Contract for ledger auditing:
 - List<Transaction> getTransactionHistory();
 - String generateStatement();

5. BankAccount.java (Abstract Class)

- Implements Transactable and Reportable.
- **Fields:**
 - private final String accountNumber
 - private final Customer owner
 - protected double balance
 - private final List<Transaction> transactions (Thread-safe or synchronized list)
 - private final Object lock = new Object(); (Dedicated lock object for fine-grained synchronization)
- **Constructor:** Validates initial deposit (amount >= 0). Rejects negative opening balances.
- **Template Methods & Overloading:**
 - Concrete implementation of deposit(double amount):
 - Rejects amounts ≤ 0 with InvalidAmountException.
 - Synchronized balance update.
 - Appends Transaction record.
 - Overloaded deposit(double amount, String note).
 - Abstract method public abstract void withdraw(double amount) throws ...; (forcing distinct child behavior).
- **Audit Protection:**
 - getTransactionHistory() MUST return Collections.unmodifiableList(new ArrayList<>(transactions)) (Defensive copy).
- **Identity Contract:**
 - Override equals(Object o): Two accounts are equal if and only if their accountNumber is identical.
 - Override hashCode(): Based on accountNumber.
 - Override toString(): Prints account summary cleanly.

6. SavingsAccount.java (Inheritance & Overriding)

- Extends BankAccount.
- **Fields:**
 - private static final double MINIMUM_BALANCE = 1000.00;

- private double interestRate; (e.g., 0.04 for 4%)
- **Behavior:**
 - Overrides withdraw(double amount):
 - Invariant: Balance cannot drop below MINIMUM_BALANCE.
 - If (balance - amount) < MINIMUM_BALANCE, throw InsufficientFundsException detailing account balance, requested amount, and the minimum balance rule.
 - Updates balance and logs transaction.
 - Method applyInterest(): Calculates interest on current balance, credits it via an INTEREST transaction.

7. CurrentAccount.java (Inheritance & Overriding)

- Extends BankAccount.
- **Fields:**
 - private double overdraftLimit; (e.g., 25000.00)
 - private static final double OVERDRAFT_FEE = 50.00;
- **Behavior:**
 - Overrides withdraw(double amount):
 - Invariant: Can withdraw up to (balance + overdraftLimit).
 - If amount > (balance + overdraftLimit), throw OverdraftLimitExceededException.
 - If amount > balance, deduct balance (allowing it to become negative) and apply OVERDRAFT_FEE as an additional recorded transaction.

B. Service Layer (BankServiceImpl.java)

- Holds state in memory using Collections:
 - private final Map<String, BankAccount> accounts = new ConcurrentHashMap<>();
 - private final Map<String, Customer> customers = new ConcurrentHashMap<>();
- Methods to implement:
 - Customer registerCustomer(String name, String email, String phone)
 - BankAccount openSavingsAccount(String customerId, double initialDeposit, double interestRate)
 - BankAccount openCurrentAccount(String customerId, double initialDeposit, double overdraftLimit)
 - BankAccount findAccount(String accountNumber) (Throws AccountNotFoundException if missing)
 - Customer findCustomer(String customerId) (Throws CustomerNotFoundException if missing)
 - void transfer(String fromAccNum, String toAccNum, double amount, String description)
 - **CRITICAL REQUIREMENT - Deadlock Prevention:**
To avoid deadlocks during concurrent inter-account transfers (A -> B while B -> A), the locks on both accounts MUST be acquired in a deterministic, globally consistent order.
 - Sort accounts by accountNumber.compareTo(...).

- Acquire lock on the lower account first, then lock on the higher account.
- Validate both accounts exist.
- Verify sufficient funds on fromAccount.
- Withdraw from fromAccount and deposit to toAccount.
- Log TRANSFER_OUT on sender and TRANSFER_IN on receiver.
- Ensure transaction atomicity: if an exception occurs mid-way, state remains uncorrupted.

C. Exception Handling Hierarchy

Every custom exception must be a checked exception (extends Exception) or appropriately structured domain exception:

1. BankingException (Superclass, takes message and optional cause)
2. InsufficientFundsException (Includes fields: double currentBalance, double requestedAmount, double minimumBalance)
3. OverdraftLimitExceededException (Includes fields: double currentBalance, double requestedAmount, double overdraftLimit)
4. AccountNotFoundException (Includes String accountNumber)
5. CustomerNotFoundException (Includes String customerId)
6. DuplicateAccountException (Includes String accountNumber)
7. InvalidAmountException (Includes double invalidAmount, message explaining why)

D. Multithreading Demonstration (concurrency/)

Create a dedicated demonstration class (ConcurrencyHarness.java) demonstrating two real-world stress scenarios:

Scenario 1: Race Condition on Shared Account (ATM vs. Online POS)

- Create one SavingsAccount with balance ₹10,000.
- Spin up 10 concurrent threads (ExecutorService), each attempting to withdraw ₹1,200 simultaneously (Total requested: ₹12,000).
- Show how the synchronized implementation allows exactly 7 or 8 successful withdrawals, safely rejecting the rest with InsufficientFundsException without leaving the balance inconsistent or below ₹1,000.

Scenario 2: High-Frequency Bidirectional Transfers (Deadlock-Free)

- Create two accounts: ACC-1 (₹20,000) and ACC-2 (₹20,000).
- Thread A transfers ₹100 from ACC-1 to ACC-2 500 times.
- Thread B transfers ₹100 from ACC-2 to ACC-1 500 times.
- Run concurrently. Verify that the system executes all 1,000 transfers without deadlock, and total pooled money remains exactly ₹40,000.

E. User Interface Layer (ui/BankApplicationCLI.java)

Build an interactive, defensive, text-based console menu:

- **Menu Options:**
 1. Register New Customer
 2. Open Savings Account
 3. Open Current Account
 4. Deposit Funds
 5. Withdraw Funds
 6. Inter-Account Transfer
 7. View Account Statement / Passbook
 8. View Customer Profile & Owned Accounts
 9. Run Multithreading & Concurrency Stress Test
 10. Exit
- **CLI Quality Requirements:**
 - Defensive reading: never crash on InputMismatchException (wrap scanner parsing with retry loops).
 - Catch all custom banking exceptions and print user-friendly, informative error messages.
 - Formatted tabular output for statements and balance summaries.

4. INSTRUCTIONS TO CODE AGENT

1. Generate **all** files without skipping any method implementations.
2. Ensure full type safety, zero raw collection types, and clean Java conventions (camelCase, JavaDoc on core methods).
3. Do not rely on external 3rd-party libraries (use pure standard Java SE library).
4. Include a root-level README.md explaining:
 - Compilation and execution commands (javac / java).
 - Explanation of how Object methods, polymorphism, encapsulation, collections, and thread-safety were implemented.
 - Walkthrough of the deadlock prevention mechanism.